Assignment Editorials: Loops & Flow Control

Q1. Operations on Numbers - Editorial

Problem Understanding

You are given a single number stored in a variable called `number`. You need to perform three operations **in sequence**:

1. Multiply the value by 3
2. Add 7 to the result
3. Subtract 10 from the result

Finally, print the updated value.

Approach

This problem tests:

1. Sequential arithmetic operations
2. Understanding order of operations
3. Updating a variable step by step

Mathematical Formula:

Result = (number × 3) + 7 - 10 = (number × 3) - 3

Step-by-Step Logic

1. **Read** the input number
2. **Multiply** the number by 3
3. **Add** 7 to the result
4. **Subtract** 10 from the result
5. **Print** the final value

Pseudo Code

```
READ number

number ← number × 3
number ← number + 7
number ← number - 10

PRINT number
```

Alternative (single expression):

```
READ number

result ← (number × 3) + 7 - 10

PRINT result
```

Simplified version:

```
READ number

result ← (number × 3) - 3

PRINT result
```

Walkthrough Example

Input: `4`

Step	Operation	Calculation	Updated Value
Initial	Read input	-	4
Step 1	Multiply by 3	4 × 3	12
Step 2	Add 7	12 + 7	19
Step 3	Subtract 10	19 - 10	9

Output: `9`

More Examples

Input	Step 1 (×3)	Step 2 (+7)	Step 3 (-10)	Output
1	3	10	0	0
4	12	19	9	9
10	30	37	27	27
20	60	67	57	57

Complexity Analysis

Type	Complexity	Explanation
Time	O(1)	Fixed number of operations (3 arithmetic operations)
Space	O(1)	Only storing one input value

Q2. Cube of a Number - Editorial

Problem Understanding

Given a number N, you need to find its cube (N raised to the power 3).

Cube of a number = Number × Number × Number = N^3

Approach

This is a basic arithmetic problem:

1. Read the input number N
2. Calculate N × N × N
3. Print the result

Step-by-Step Logic

1. **Read** the number N
2. **Calculate** cube = N × N × N
3. **Print** the result

Pseudo Code

```
READ N

cube ← N × N × N

PRINT cube
```

Alternative (using exponentiation):

```
READ N

cube ← N ^ 3

PRINT cube
```

Dry Run

Example:

Input: `3`

Step	Operation	Result
Read	N = 3	3
Calculate	3 × 3 × 3	27

Output: `27`

Complexity Analysis

Type	Complexity	Explanation
Time	O(1)	Fixed number of multiplications
Space	O(1)	Only storing input and result

Q3. Product of Six I - Editorial

Problem Understanding

You are given 6 numbers stored in variables: `one`, `two`, `three`, `four`, `five`, `six`. Your task is to find the product (multiplication) of all six numbers.

Product = one × two × three × four × five × six

Approach

1. Read all 6 input numbers
2. Multiply all of them together
3. Print the final product

Step-by-Step Logic

1. **Read** all 6 numbers from input
2. **Multiply** all numbers together
3. **Print** the product

Pseudo Code

```
READ one, two, three, four, five, six

product ← one × two × three × four × five × six

PRINT product
```

Alternative (iterative approach):

```
READ one, two, three, four, five, six

product ← 1
product ← product × one
product ← product × two
product ← product × three
product ← product × four
product ← product × five
product ← product × six

PRINT product
```

Dry Run

Input: `1 2 3 4 5 6`

Step	Operation	Running Product
Start	Initialize	1
× one	1 × 1	1
× two	1 × 2	2
× three	2 × 3	6
× four	6 × 4	24
× five	24 × 5	120
× six	120 × 6	720

Output: `720`

Complexity Analysis

Type	Complexity	Explanation
Time	O(1)	Fixed number of operations (5 multiplications)
Space	O(1)	Only storing 6 input values and one result

Q4. Products of Numbers II - Editorial

Problem Understanding

Given three integers A, B, and C, you need to calculate the expression:

Expression = (A × B) + C

First multiply A and B, then add C to the result.

Approach

1. Read the three input numbers A, B, and C
2. Multiply A and B
3. Add C to the product
4. Print the result

Step-by-Step Logic

1. **Read** A, B, C from input
2. **Calculate** product = A × B
3. **Calculate** result = product + C
4. **Print** result

Pseudo Code

```
READ A, B, C

product ← A × B
result ← product + C

PRINT result
```

Alternative (single expression):

```
READ A, B, C

result ← (A × B) + C

PRINT result
```

Dry Run

Input: `3 4 5`

Step	Operation	Value
Read	A=3, B=4, C=5	-
Multiply	A × B = 3 × 4	12
Add	12 + C = 12 + 5	17

Output: `17`

Complexity Analysis

Type	Complexity	Explanation
Time	O(1)	Two arithmetic operations
Space	O(1)	Only storing 3 input values and result

Q5. Divisibly by K - Editorial

Problem Understanding

Given two numbers `num` and `K`, find all numbers in the range [1, num] that are divisible by K.

A number `i` is divisible by `K` if `i % K == 0` (i.e., remainder is zero when divided by K).

Approach

1. Read num and K
2. Iterate through all numbers from 1 to num
3. For each number, check if it's divisible by K
4. If divisible, print it on a new line

Step-by-Step Logic

1. **Read** num and K from input
2. **Loop** through numbers from 1 to num (inclusive)
3. **Check** if current number % K == 0
4. **Print** the number if condition is true

Pseudo Code

```
READ num, K

FOR i FROM 1 TO num DO
    IF i % K == 0 THEN
        PRINT i
```

Alternative (direct multiples approach):

```
READ num, K

i ← K
WHILE i <= num DO
    PRINT i
    i ← i + K
```

Dry Run

Input: `7 2`

i	i % K (i % 2)	Divisible?	Output
1	1 % 2 = 1	❌ No	-
2	2 % 2 = 0	✅ Yes	2
3	3 % 2 = 1	❌ No	-
4	4 % 2 = 0	✅ Yes	4
5	5 % 2 = 1	❌ No	-
6	6 % 2 = 0	✅ Yes	6
7	7 % 2 = 1	❌ No	-

Output:

```
2
4
6
```

Complexity Analysis

Type	Complexity	Explanation
Time	O(num)	Loop runs from 1 to num
Space	O(1)	Only storing loop variable

Note: Alternative approach has O(num/K) time complexity as it directly jumps to multiples.

Q6. Divisible by K II - Editorial

Problem Understanding

Given two numbers `num` and `K`, find the **sum** of all numbers in the range [1, num] that are divisible by K.

A number `i` is divisible by `K` if `i % K == 0` (remainder is zero).

Approach

1. Read num and K
2. Initialize a sum variable to 0
3. Iterate through all numbers from 1 to num
4. For each number divisible by K, add it to sum
5. Print the final sum

Step-by-Step Logic

1. **Read** num and K from input
2. **Initialize** sum = 0
3. **Loop** through numbers from 1 to num
4. **Check** if current number % K == 0
5. **Add** the number to sum if divisible
6. **Print** the total sum

Pseudo Code

```
READ num, K

sum ← 0

FOR i FROM 1 TO num DO
    IF i % K == 0 THEN
        sum ← sum + i

PRINT sum
```

Alternative (direct multiples approach):

```
READ num, K

sum ← 0
i ← K

WHILE i <= num DO
    sum ← sum + i
    i ← i + K

PRINT sum
```

Dry Run

Input: `7 2`

i	i % K (i % 2)	Divisible?	sum
-	-	Initial	0
1	1	❌ No	0
2	0	✅ Yes	0 + 2 = 2
3	1	❌ No	2
4	0	✅ Yes	2 + 4 = 6
5	1	❌ No	6
6	0	✅ Yes	6 + 6 = 12
7	1	❌ No	12

Output: `12`

Complexity Analysis

Type	Complexity	Explanation
Time	O(num)	Loop runs from 1 to num
Space	O(1)	Only storing sum and loop variable

Note: Alternative approach has O(num/K) time complexity.

Q7. From A to B - Editorial

Problem Understanding

Given two numbers `max` and `min`, print all numbers starting from `min` up to (but not including) `max`. Each number should be printed on a new line.

Range: [min, max) — includes min, excludes max

Approach

1. Read max and min values
2. Loop from min to max-1 (or until i < max)
3. Print each number on a new line

Step-by-Step Logic

1. **Read** max and min from input
2. **Loop** starting from min, continue while i < max
3. **Print** each value of i
4. **Increment** i by 1

Pseudo Code

```
READ max, min

FOR i FROM min TO max-1 DO
    PRINT i
```

Alternative (while loop):

```
READ max, min

i ← min
WHILE i < max DO
    PRINT i
    i ← i + 1
```

Dry Run

Input: `19 13`

i	i < max (i < 19)?	Output
13	✅ Yes	13
14	✅ Yes	14
15	✅ Yes	15
16	✅ Yes	16
17	✅ Yes	17
18	✅ Yes	18
19	❌ No	(stop)

Output:

```
13
14
15
16
17
18
```

Complexity Analysis

Type	Complexity	Explanation
Time	O(max - min)	Loop runs (max - min) times
Space	O(1)	Only storing loop variable

Q8. Range of 2 - Editorial

Problem Understanding

Given two numbers `left` and `right`, print a sequence starting from `left` where each subsequent number is **double** the previous one. Continue until the number exceeds `right`.

Sequence: left, left×2, left×4, left×8, ... (while ≤ right)

Approach

1. Read left and right values
2. Start with current = left
3. While current ≤ right, print and double the value
4. Stop when current exceeds right

Step-by-Step Logic

1. **Read** left and right from input
2. **Initialize** current = left
3. **Loop** while current ≤ right:
    - Print current
    - Double current (current = current × 2)

Pseudo Code

```
READ left, right

current ← left

WHILE current <= right DO
    PRINT current
    current ← current × 2
```

Dry Run

Input: `4 10`

Iteration	current	current ≤ right?	Action
1	4	4 ≤ 10 ✅	Print 4, current = 4×2 = 8
2	8	8 ≤ 10 ✅	Print 8, current = 8×2 = 16
3	16	16 ≤ 10 ❌	Stop

Output:

```
4
8
```

Complexity Analysis

Type	Complexity	Explanation
Time	O(log(right/left))	Value doubles each iteration, so iterations = $\log_2$(right/left)
Space	O(1)	Only storing current variable